Table of contents

Appendix A — The Docker Compose files, line by line

The main chapters keep the `compose.yaml` files terse on purpose — you don't need to understand every line to get a working homelab. This appendix is the opposite: it explains **every directive** in every Compose file the series creates, so when you come back in six months you can read your own stack and know exactly what each line does (and what's safe to change).

By the end of the series your Pi runs five containers across three Compose projects plus one `docker run`:

Project folder	File	Containers
`~/pihole`	`compose.yaml`	`pihole`
`~/proxy`	`compose.yaml`	`caddy`
`~/dashboard`	`compose.yaml`	`homepage`, `portainer`
(none — `docker run`)	—	`audiobookshelf`

Compose concepts that show up everywhere

Before the per-file walkthroughs, here are the directives you'll see repeatedly. Read this once and the individual files become obvious.

- **`services:`** — the top-level map. Each key under it (e.g. `pihole:`) is one container Compose will manage.
- **`image:`** — which prebuilt image to pull and run, in `repository:tag` form. `pihole/pihole:latest` means "the `latest` tag of the `pihole/pihole` image." `:latest` floats; pinning a version (e.g. `:2024.07.0`) is more reproducible but you update by hand. This guide uses `:latest` for simplicity and updates with `docker compose pull`.
- **`container_name:`** — the fixed, human-friendly name (e.g. `pihole`). Without it, Compose auto-generates names like `pihole-pihole-1`. We set it explicitly because **the reverse proxy and the inter-container networking address each service by this exact name** — `reverse_proxy pihole:80` only works if the container is literally named `pihole`.
- **`ports:`** — publishes a **host** port to a **container** port, written `"HOST:CONTAINER"`. `"8081:80"` means "traffic arriving on the Pi's port 8081 is forwarded to port 80 inside the container." A bare `"80:80"` binds **all** the host's network interfaces (LAN *and* tailnet); prefixing an address, `"192.168.1.50:80:80"`, binds only that one interface. Ports are only needed for traffic entering from *outside* Docker — containers on the same Docker network reach each other directly without any `ports:` entry.

- `environment:` — variables set inside the container. This is how most images are configured. Values can be literals or `${VAR}` references resolved from the project's `.env` file (see below).
- `volumes:` — persistent storage. Two forms appear in this series:
  - **Bind mount** — `./etc-pihole:/etc/pihole` maps a *folder on the Pi* to a path inside the container. You can see and back up the files directly. The `./` is relative to the folder the `compose.yaml` lives in. A `:ro` suffix (`...:/etc/caddy/Caddyfile:ro`) mounts it **read-only**.
  - **Named volume** — `portainer_data:/data` stores data in a Docker-managed volume (declared in the top-level `volumes:` block). Use it when you don't need to touch the files yourself, only persist them across container recreations.
- `networks:` — which Docker networks the container joins. Containers on the same network resolve each other **by container name** as a hostname. This is the backbone of the reverse proxy: Caddy and the services share the `homelab` network, so `pihole`, `audiobookshelf`, and `homepage` are resolvable names.
- `restart:` — the restart policy. `unless-stopped` restarts the container on crash and on boot, *unless* you deliberately stopped it. `always` is similar but restarts even one you stopped, after a Docker daemon restart. Both are correct for an always-on Pi; the difference rarely matters.
- **Top-level `networks:` / `volumes:`** — declare resources the services reference. `external: true` means "this network already exists; don't create or destroy it, just attach to it" — used for the shared `homelab` network that outlives any single project.

i `${VAR}` interpolation, `.env`, and the `:?` / `:-` forms

Compose substitutes `${VAR}` from a file named `.env` in the same folder *before* starting the container. This is the mechanism that keeps secrets out of the file you commit:

- `${PIHOLE_PASSWORD}` — plain substitution; empty if unset.
- `${PIHOLE_PASSWORD:?set PIHOLE_PASSWORD in .env}` — **required**: Compose aborts with that error message if the variable is missing or empty. Used for secrets you must not launch without.
- `${ABS_TOKEN:-}` — **default if unset**: the part after `:-` is the fallback (here, empty string), so a missing optional token doesn't error.

The real values live in `.env`, which is listed in `.gitignore` and never leaves the Pi. The `compose.yaml` holds only references, so it's safe to publish.

`~/pihole/compose.yaml` — Pi-hole

```yaml
services:
  pihole:
    container_name: pihole
    image: pihole/pihole:latest

    ports:
      - "53:53/tcp"
```

```
      - "53:53/udp"
      - "8081:80/tcp"

    environment:
      TZ: "${TZ}"
      FTLCONF_webserver_api_password: "${PIHOLE_PASSWORD:?set
PIHOLE_PASSWORD in .env}"
      FTLCONF_dns_upstreams: "1.1.1.1;1.0.0.1"
      FTLCONF_dns_listeningMode: "ALL"

    volumes:
      - ./etc-pihole:/etc/pihole

    networks:
      - homelab

    restart: unless-stopped

networks:
  homelab:
    external: true
```

- **`ports: 53:53/tcp and 53:53/udp`** — DNS uses **both** transports (UDP for almost everything, TCP for large responses), so both are published. They're bound to all interfaces so the Pi answers DNS for your LAN *and* over the tailnet. (In Part 3, before the proxy existed, these were bound to the Pi's LAN IP only — `${PIHOLE_IP}:53:53`; Part 4 widens them.)
- **`ports: 8081:80/tcp`** — Pi-hole's web UI listens on port 80 *inside* the container; we publish it on the host as **8081** so the host's port 80 stays free for Caddy.
- **`TZ`** — the timezone, so Pi-hole's logs and graphs use your local time.
- **`FTLCONF_*`** — Pi-hole v6's configuration system. Each variable maps to a key in `/etc/pihole/pihole.toml` by the rule `FTLCONF_<section>_<key>` → `[section] key`. Anything set this way is **re-applied on every container start** and shows as read-only in the web UI — exactly what you want for file-defined, reproducible config:
  - `FTLCONF_webserver_api_password` — the admin/API password (the v6 successor to the old `WEBPASSWORD`).
  - `FTLCONF_dns_upstreams` — the real resolvers Pi-hole forwards *non-blocked* queries to; `1.1.1.1;1.0.0.1` is Cloudflare's primary + secondary.
  - `FTLCONF_dns_listeningMode: "ALL"` — answer queries on any interface. Required for a Dockerized Pi-hole, because LAN clients' queries arrive *through* the Docker gateway and the stricter "local-only" mode would reject them.
- **`volumes: ./etc-pihole:/etc/pihole`** — persists Pi-hole's entire state (config, blocklists, query database, your local DNS records) in a folder next to the compose file, so recreating the container loses nothing.
- **`networks: homelab` + top-level `external: true`** — joins the shared proxy network *declaratively*. This is deliberate: attaching it by hand with `docker network connect` would be wiped by the next `--force-recreate` (see the warning in Part 4). In the file, it always

re-attaches.

`~/proxy/compose.yaml` + `Caddyfile` — the reverse proxy

```yaml
services:
  caddy:
    container_name: caddy
    image: caddy:latest
    ports:
      - "80:80"
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data
      - caddy_config:/config
    networks:
      - homelab
    restart: unless-stopped

networks:
  homelab:
    external: true

volumes:
  caddy_data:
  caddy_config:
```

- **`ports: 80:80`** — Caddy is the **single front door**. It owns the host's port 80 on all interfaces, which is why no other container may publish port 80. Every `http://*.home` request lands here first.
- **`volumes`**:
  - `./Caddyfile:/etc/caddy/Caddyfile:ro` — mounts your routing table into the container, **read-only** (Caddy never needs to write its own config).
  - `caddy_data` / `caddy_config` — named volumes for Caddy's internal state (most importantly any TLS certificates, if you later use a real domain). Kept as named volumes because you never edit them by hand.
- **`networks: homelab`** — lets Caddy reach each backend by container name. This is the whole trick: Caddy connects to `pihole:80`, `audiobookshelf:80`, `homepage:3000` *over this network*, completely ignoring the host port mappings. That's why moving Pi-hole's host UI port to 8081 doesn't affect Caddy.

The companion **`Caddyfile`** is the routing table:

```
http://pihole.home {
    redir / /admin 302
    reverse_proxy pihole:80
```

```
}

http://abs.home {
    reverse_proxy audiobookshelf:80
}

http://home.home, http://homelab {
    reverse_proxy homepage:3000
}
```

- **`http://` prefix** — tells Caddy to serve plain HTTP and **not** try to fetch a TLS certificate. There's no public certificate authority for a private `.home` name, so without this prefix Caddy would loop trying (and failing) to get one. On your authenticated tailnet, plain HTTP is fine.
- **`{ ... }` site block** — one per hostname (or comma-separated list of hostnames). Caddy matches the incoming request's `Host` header to the right block.
- **`reverse_proxy <name>:<port>`** — forwards the request to that container over the `homelab` network. The port is the service's *internal* port, not its published host port.
- **`redir / /admin 302`** — Pi-hole's UI lives under `/admin`; this sends a bare `http://pihole.home/` to `http://pihole.home/admin` so you land in the right place. The `302` is a temporary-redirect status code.
- **`http://home.home, http://homelab`** — two names, one backend: both open the dashboard.

`~/dashboard/compose.yaml` — Homepage + Portainer

```
services:
  homepage:
    image: ghcr.io/gethomepage/homepage:latest
    container_name: homepage
    volumes:
      - ./config:/app/config
      - /var/run/docker.sock:/var/run/docker.sock:ro
    environment:
      HOMEPAGE_ALLOWED_HOSTS: home.home,homelab,homelab.your-tailnet.ts.net
      HOMEPAGE_VAR_PIHOLE_PASSWORD: ${PIHOLE_PASSWORD:?set PIHOLE_PASSWORD
in .env}
      HOMEPAGE_VAR_ABS_TOKEN: ${ABS_TOKEN:-}
    networks:
      - homelab
    restart: unless-stopped

  portainer:
    image: portainer/portainer-ce:lts
    container_name: portainer
    ports:
      - "9443:9443"
```

```
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - portainer_data:/data
    networks:
      - homelab
    restart: always

networks:
  homelab:
    external: true

volumes:
  portainer_data:
```

This is the only project with **two** services. Note Homepage has **no `ports:`** — it's reached only through Caddy (`homepage:3000` over `homelab`), so it never touches a host port.

Homepage:

- **`image: ghcr.io/gethomepage/homepage:latest`** — the full registry path; this one lives on GitHub's Container Registry (`ghcr.io`) rather than Docker Hub.
- **`volumes`**:
  - `./config:/app/config` — your dashboard's YAML config (tiles, widgets, bookmarks) on the Pi, editable directly.
  - `/var/run/docker.sock:/var/run/docker.sock:ro` — the **Docker socket**, **read-only**. This lets Homepage read container status (the live "running" dot, CPU/RAM) without being able to control Docker.
- **`HOMEPAGE_ALLOWED_HOSTS`** — a required allow-list of hostnames Homepage will render for. It must contain *every* name Caddy forwards here (`home.home`, `homelab`, and the Pi's full tailnet name). A request for a name not on the list gets a blank page — the single most common Homepage gotcha. **This variable only takes effect on a container recreate** (`up -d --force-recreate`), not a plain `restart`.
- **`HOMEPAGE_VAR_*`** — Homepage exposes any `HOMEPAGE_VAR_<NAME>` variable to its config files as `{{HOMEPAGE_VAR_<NAME>}}`. This is how widget secrets (the Pi-hole password, the Audiobookshelf API token) reach the config **without being written into the config files** — they come from `.env` at runtime.

Portainer:

- **`image: portainer/portainer-ce:lts`** — Community Edition, Long-Term Support tag. Runs natively on the 64-bit Pi.
- **`ports: 9443:9443`** — Portainer's own HTTPS UI (self-signed certificate). It keeps a host port because you reach it directly at `https://homelab:9443`, not through Caddy.
- **`volumes`**:
  - `/var/run/docker.sock:/var/run/docker.sock` — the Docker socket, **read-write this time** (no `:ro`). Portainer's whole job is to *control* Docker — start, stop, recreate, pull — so it needs full socket access.

  - `portainer_data:/data` — a named volume for Portainer's own database (users, settings).

- **`restart: always`** — Portainer is your break-glass management console; you want it back even after a manual stop + daemon restart.

> ⚠ The Docker socket is root-equivalent
>
> Mounting `/var/run/docker.sock` into a container gives that container control over Docker — which on Linux is effectively root on the host. That's why Homepage gets it `:ro` (it only needs to *read* status) and only Portainer gets it read-write (it needs to *manage*). Never expose a container that mounts the socket to the public internet; on your tailnet-only homelab it's fine.

Audiobookshelf — the `docker run` hold-out

Audiobookshelf is the one service launched with `docker run` instead of Compose, because Part 2 introduces it before any Compose project exists:

```
docker run -d \
  --name audiobookshelf \
  --restart unless-stopped \
  -p 13378:80 \
  -v ~/Audiobooks:/audiobooks \
  -v ~/Audiobooks-drill:/audiobooks-drill \
  -v ~/.config/audiobookshelf/config:/config \
  -v ~/.config/audiobookshelf/metadata:/metadata \
  ghcr.io/advplyr/audiobookshelf:latest
```

The flags map one-to-one onto the Compose directives above:

`docker run` flag	Compose equivalent	Meaning
`-d`	*(default)*	Detached — run in the background.
`--name audiobookshelf`	`container_name:`	Fixed name (so `abs.home` can proxy to it).
`--restart unless-stopped`	`restart:`	Auto-start on boot and after crashes.
`-p 13378:80`	`ports:`	Publish host 13378 → container 80.
`-v host:container`	`volumes:`	Bind-mount audio folders, config, metadata.

Why this one isn't on the `homelab` network in its run command

The `homelab` network doesn't exist until Part 4, which is *after* Audiobookshelf is created — so Part 4 attaches it with `docker network connect homelab audiobookshelf`. Because it's not a Compose service, that imperative attachment sticks **until you recreate the container** (e.g. the update procedure in Part 2's *Maintenance*). After any `docker rm`/re-`run`, re-attach it with `docker network connect homelab audiobookshelf`, or `abs.home` will stop resolving through Caddy. (If you prefer, you can convert Audiobookshelf to its own `~/audiobookshelf/compose.yaml` using the patterns above, with a `networks: [homelab]` block, to make it as recreate-proof as the others.)